Cheatsheet · 9 chapters on one page

API Automation Testing cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

96 key ideas | 45 formulas | 43 tables | 113 tips and traps

STEP 1 Java

1.1 Java · Rest Assured

The 20 lessons of the Java · Rest Assured track.

KEY IDEAS

- **What API automation is and what to check** An API test sends a real HTTP request to a service and checks the response, with no browser involved.
- **Project setup with Maven** Create a Maven project and add the dependencies below. Test code lives in src/test/java , and files such as JSON schemas live in src/test/resources .
- **Your first test: GET a booking** This test asks for booking 1 and checks three things: the status, the content type, and that the first name is a non empty string.
- **Building requests: params, headers and cookies** Real requests carry more than a path. Rest Assured gives a method for each part of the request.
- **Create, update and delete (CRUD)** There are three common ways to send a JSON body: a text block, a Map , or a Java object that Jackson turns into JSON. All three produce the same request.
- **Assertions with Hamcrest and GPath** Rest Assured reads values with GPath , the path language from Groovy, and compares them with Hamcrest matchers. A few patterns cover almost every check.
- **Extracting values and chaining calls** Most real flows need a value from one response in the next request: a token, a new id, a link. Rest Assured gives three ways to pull values out.
- **Authentication: token cookie, Basic, Bearer, OAuth 2 and API keys** APIs protect data in a few standard ways. Restful Booker accepts a token cookie or Basic auth. Most other APIs you test will use a Bearer token, an API key or OAuth 2.
- **Reusable specifications and filters** Copying the same base URI, headers and checks into every test makes a suite hard to change. A RequestSpecification holds shared request settings and a ResponseSpecification holds shared checks.
- **Data driven tests with DataProvider** One test method can run with many rows of data. TestNG feeds each row from a @DataProvider . This is how you cover boundary values and invalid inputs without copying tests.
- **JSON Schema validation** A JSON Schema describes the shape of a response: which fields exist, their types, which are required, and allowed patterns.
- **POJOs, records and serialization** Turning a Java object into JSON is serialization . Turning JSON back into an object is deserialization . Rest Assured does both with Jackson whenever it is on the classpath. Java records make the model classes short.
- **Designing a real framework** Tests that call given() directly are fine for twenty tests. For hundreds, split the code into layers so that a change to an endpoint is fixed in one place. This is the API version of the Page Object pattern.
- **Mocking other services with WireMock** Sometimes the API you test calls another service that is slow, costly or not built yet, such as a payment gateway. WireMock starts a fake HTTP server that returns answers you define.
- **Parallel runs, reports and CI** A mature suite runs on every pull request, in parallel, and publishes a report. TestNG handles parallel runs through testng.xml .

- **Expert topics: polling, uploads, GraphQL, contracts and more** Some APIs process work in the background, so the result appears a few seconds later. Never use Thread.sleep . Poll with a time limit using Awaitility.
- **Real project essentials: environments, secrets, fake data, safe logging and cleanup** In a company, the same Rest Assured suite runs on a laptop, in QA and in staging. Passwords come from the CI secret store, every run makes its own realistic data, failing calls are logged with the token hidden, and ...
- **Real world scenario: a hotel front desk day** A guest calls the front desk. The receptionist books a room, the guest asks for breakfast to be added, then the trip is cancelled.

FORMULAS

Project layout

```
booker-api-tests/
   ├── pom.xml
   └── src/test/
      ├── java/tests/
      │   ├── FirstTest.java
      │   ├── CrudTest.java
      │   │   └── ...
      └── resources/schemas/
         └── booking-schema.json
```

Lesson: Project setup with Maven

A negative auth test

```
@Test
public void updateWithoutTokenIsForbidden() {
given()
.contentType(ContentType.JSON)
.body(Map.of("firstname", "Hacker"))
.when()
.patch("/booking/1")
.then()
.statusCode(403);


when().get("/booking/1").then().body("firstname",
not(equalTo("Hacker")));
}
```

Lesson: Authentication: token cookie, Basic, Bearer, OAuth 2 and API keys

src/test/java/tests/model/CreatedBooking.java

```
package tests.model;

import
com.fasterxml.jackson.annotation.JsonIgnoreProper
ties;

@JsonIgnoreProperties(ignoreUnknown = true)
public record CreatedBooking(int bookingid,
Booking booking) {}
```

Lesson: POJOs, records and serialization

Extra dependency

```
<dependency>
<groupId>org.wiremock</groupId>
<artifactId>wiremock-standalone</artifactId>
<version>3.13.2</version>
<scope>test</scope>
</dependency>
```

Lesson: Mocking other services with WireMock

src/test/resources/testng.xml

```
<!DOCTYPE suite SYSTEM
"https://testng.org/testng-1.0.dtd">
<suite name="Booker API" parallel="classes"
thread-count="4">
<test name="smoke">
<groups><run><include name="smoke"/></run>
</groups>
<packages><package name="tests"/></packages>
</test>
</suite>
```

Lesson: Parallel runs, reports and CI

Poll until a job is done

```
import static org.awaitility.Awaitility.await;
import java.time.Duration;

await().atMost(Duration.ofSeconds(30))
.pollInterval(Duration.ofSeconds(1))
.until(() → get("/jobs/{id}",
jobId).path("status").equals("DONE"));
```

Lesson: Expert topics: polling, uploads, GraphQL, contracts and more

Add DataFaker to pom.xml

```
<dependency>
<groupId>net.datafaker</groupId>
<artifactId>datafaker</artifactId>
<version>2.7.0</version>
<scope>test</scope>
</dependency>
```

Lesson: Real project essentials: environments, secrets, fake data, safe logging and cleanup

TABLES

Table: check · example on restful booker · why

Check	Example on Restful Booker	Why
Status code	`GET /booking/1` gives 200	The first sign that the call worked
Body values	`firstname` is `Sally`	Proves the right data came back
Body shape	Every booking has `bookingdates.checkin`	Catches renamed or missing fields
Headers	`Content-Type` contains `application/json`	Clients parse based on it
Negative cases	PUT without a token gives 403	Security and validation rules hold
Time	Answer in under 2 seconds	Catches slow endpoints early

Lesson: What API automation is and what to check

Table: part of the request · rest assured method · example

Part of the request	Rest Assured method	Example
Path parameter	`pathParam("id", 1)`	`/booking/{id}`
Query parameter	`queryParam("firstname", "Sally")`	`?firstname=Sally`
Header	`header("Accept", "application/json")`	Any header
Cookie	`cookie("token", token)`	Session or token cookie
Form field	`formParam("user", "admin")`	HTML style forms
Content type	`contentType(ContentType.JSON)`	Sets the Content-Type header

Lesson: Building requests: params, headers and cookies

Table: method · meaning · restful booker answer

Method	Meaning	Restful Booker answer
`POST`	Create something new	200 with `bookingid`
`GET`	Read	200, or 404 when missing
`PUT`	Replace every field	200, needs auth
`PATCH`	Change only the fields you send	200, needs auth
`DELETE`	Remove	201, needs auth

Lesson: Create, update and delete (CRUD)

Table: gpath · reads · on this json

GPath	Reads	On this JSON
`firstname`	A top level field	a booking
`bookingdates.checkin`	A nested field	a booking
`[0].bookingid` or `bookingid[0]`	An item in a top level array	the booking list
`bookingid`	One field from every item	the booking list
`size()`	How many items	the booking list
`findAll { it.bookingid > 5 }.bookingid`	A filter	the booking list
`max { it.bookingid }.bookingid`	The largest	the booking list

Lesson: Assertions with Hamcrest and GPath

Table: habit · how this class does it

Habit	How this class does it
One suite, many servers	`TestConfig.baseUrl()` reads `-Denv`, `TEST_ENV` or `-DbaseUrl`
Secrets out of Git	password from `-DapiPassword` or `API_PASSWORD`, a default only for the public practice API
Realistic unique data	DataFaker with the Indian English locale
Logs that explain failures but hide secrets	`enableLoggingOfRequestAndResponseIfValidationFails()` and `blacklistHeader`
Leave QA clean	`@AfterMethod(alwaysRun = true)` deletes everything the test created

Lesson: Real project essentials: environments, secrets, fake data, safe logging and cleanup

Table: release · what changed · how it helps an api tester

Release	What changed	How it helps an API tester
6.1.0 (October 2026)	HTTP QUERY method, JSON Schema drafts 6, 7, 2019-09 and 2020-12, `java.time` values as parameters, Authorization and Cookie headers dropped on cross host redirects	Test search APIs that use QUERY, validate OpenAPI 3.1 schemas, pass dates without formatting them by hand, and stop tokens leaking to other hosts
6.2.0 (October 2026)	Core module in pure Java with no direct Groovy dependency, safer static config in parallel runs, `auth().certificate(..)` sends client certificates for mutual TLS, clearer error messages	Fewer odd failures in parallel TestNG runs, and banking style mTLS APIs from the same framework
6.3.0 (October 2026)	Prepares for 7.0: `Duration` timeouts in `HttpClientConfig`, `SSLContext` in `SSLConfig`, a migration recipe module (`rest-assured-openrewrite`)	Move off deprecated APIs now with an automatic recipe, before 7.0 removes Apache HttpClient 4 and Groovy from the defaults

Lesson: What is new in Rest Assured 6, and how it helps at work

TIPS AND TRAPS

IN THE EXAM In a Selenium or Playwright UI framework, API calls are the fastest way to create test data. Create a booking with one POST, then open the UI and check it shows up. This cuts minutes from every UI test.

Lesson: What API automation is and what to check

REMEMBER Run everything with `mvn test`. Run one class with `mvn test -Dtest=FirstTest`. Point the suite at another server with `mvn test -DbaseUrl=http://localhost:3001`, which the examples below read with `System.getProperty`.

Lesson: Project setup with Maven

REMEMBER The list endpoint returns `[{"bookingid":1},{"bookingid":2}]`. On a JSON array, the GPath `bookingid` collects the field from every item, so `everyItem(greaterThan(0))` checks them all at once.

Lesson: Building requests: params, headers and cookies

COMMON MISTAKE `.auth().basic()` waits for the server to ask for a password with a 401 before it sends one. Restful Booker answers 403 instead, so the password is never sent. Use `.auth().preemptive().basic()`, which sends it on the first request.

Lesson: Create, update and delete (CRUD)

COMMON MISTAKE JSON numbers arrive as `Integer`, `Long`, `Float` or `Double` depending on their size. `equalTo(300)` passes for 300 but `equalTo(300L)` fails. For money, check with `closeTo` or extract as `BigDecimal` through a POJO.

Lesson: Assertions with Hamcrest and GPath

IN THE EXAM Chained tests should create their own data. Do not depend on booking 1 existing on a shared server, because someone else may delete it. Create, use, then clean up in an `@AfterMethod`.

Lesson: Extracting values and chaining calls

COMMON MISTAKE Never put real passwords or tokens in code. Read them from environment variables or a secrets store, and keep them out of logs. The blacklist filter in the framework lesson hides them in Rest Assured logs.

Lesson: Authentication: token cookie, Basic, Bearer, OAuth 2 and API keys

COMMON MISTAKE Setting `RestAssured.baseURI` or `RestAssured.requestSpecification` changes a global value. That is fine for a single thread, but in parallel runs each test should pass its own spec with `given().spec(spec)`.

Lesson: Reusable specifications and filters

IN THE EXAM Restful Booker answers **500** when a required field is missing. A good API should answer **400 Bad Request** with a message. Finding gaps like this is exactly the kind of bug API testers report.

Lesson: Data driven tests with DataProvider

REMEMBER Data can also come from a CSV, JSON or Excel file. Read the file inside the `@DataProvider` method and return the rows. Add `parallel = true` to the annotation to run rows at the same time.

Lesson: Data driven tests with DataProvider

REMEMBER `"additionalProperties": false` makes the test fail when the API adds an unexpected field. Turn it on for strict contracts, and leave it off when the API is allowed to grow.

Lesson: JSON Schema validation

REMEMBER If the developers rename `totalprice` to `totalPrice`, a value check on another field still passes. The schema check fails straight away and names the missing field.

Lesson: JSON Schema validation

REMEMBER Records give you `equals` for free, so `assertEquals(read, sent)` compares every field in one line. For lists use `extract().as(new TypeRef<List<Booking>>() {})`.

Lesson: POJOs, records and serialization

REMEMBER Stubs make error paths easy to test: timeouts with `withFixedDelay`, 500 errors, broken JSON, or a dropped connection with `withFault(Fault.CONNECTION_RESET_BY_PEER)`.

Lesson: Mocking other services with WireMock

COMMON MISTAKE Logs end up in CI artifacts that many people can read. Always blacklist `Authorization`, `Cookie` and any custom API key header before you turn request logging on.

Lesson: Real project essentials: environments, secrets, fake data, safe logging and cleanup

REMEMBER

Restful Booker does not support QUERY, so that last example is for APIs that do. When your team adopts QUERY, test that it never changes data, that repeating it gives the same answer, and that gateways and caches let it through.

Lesson: What is new in Rest Assured 6, and how it helps at work

IN THE EXAM

The unique guest name makes the search step reliable on a shared server where other people create bookings at the same time.

Lesson: Real world scenario: a hotel front desk day

1.2 Java · Playwright

The 17 lessons of the Java · Playwright track.

KEY IDEAS

- **Why test APIs with Playwright** An API test sends an HTTP request and checks the answer, without a browser. Playwright makes this possible with playwright.request().newContext() , which opens no browser at all.
- **Project setup with Maven and JUnit** Run the suite with mvn test , one class with mvn test -Dtest=FirstTest , and point it at another server with mvn test -DbaseUrl=http://localhost:3001 .
- **Your first test: GET a booking** Create one Playwright and one APIRequestContext for the test class, use it in every test, and close both at the end. The context keeps the base URL, default headers and cookies.
- **Building requests with RequestOptions** Every call takes an optional RequestOptions . It holds query parameters, headers, the body, timeouts and more.
- **Create, update and delete (CRUD)** setData accepts a Map , which Playwright turns into JSON and labels with Content-Type: application/json . A String is sent as it is, so add the header yourself.
- **Assertions and reading JSON** Playwright gives the body as text or bytes. Parse it with Jackson into a JsonNode and walk it with JSON Pointer paths like /bookingdates/checkin . JUnit assertAll runs every check and reports all failures together.
- **Chaining calls and authentication** Log in, keep the token, and send it with later calls. Restful Booker wants the token in a cookie. Most other APIs want a Bearer token in the Authorization header.
- **Data driven tests with @ParameterizedTest** JUnit runs one method once per row of data. Use @CsvSource for small tables, @MethodSource for objects, and @CsvFileSource for a file.
- **JSON Schema validation** A schema describes the shape of a response. The networknt validator checks a response against it and lists every problem it finds.
- **Framework design: a client class and a base test** Move HTTP details into a client class with one method per endpoint, and move setup into a base class. Tests then only describe behaviour.
- **Mixing UI and API in one test** This is where Playwright shines for QA. Each browser context has its own request() that shares cookies with the pages in it. Three patterns save the most time.
- **Parallel runs, thread safety and CI** JUnit can run test classes in parallel. Playwright objects are not thread safe , so every thread needs its own Playwright and context.
- **Real project essentials: environments, secrets, test data, logging and cleanup** A real Playwright for Java suite runs in several places, must not print secrets, creates unique data so parallel CI jobs never clash, says exactly what went wrong when it fails, and removes what it created.
- **Real world scenario: a hotel front desk day** The receptionist logs in, books a room, adds breakfast, and later cancels. The test follows the same steps and includes the check a hacker would try.

FORMULAS

assertThat on APIResponse

```
import static
com.microsoft.playwright.assertions.PlaywrightAss
ertions.assertThat;

  assertThat(request.get("/booking/1")).isOK();
assertThat(request.get("/booking/999999")).not().
isOK();
```

Lesson: Assertions and reading JSON

A negative test

```
@Test
    void updateWithoutTokenIsForbidden() {
APIResponse res =
    anonymous.patch("/booking/1",

RequestOptions.create().setData(Map.of("firstname
", "Hacker")));
    assertEquals(403, res.status());

assertFalse(anonymous.get("/booking/1").text().co
ntains("Hacker"));
}
```

Lesson: Chaining calls and authentication

Mock what the page receives from the API

```
page.route("**/api/bookings", route →
route.fulfill(new Route.FulfillOptions()
.setStatus(500)
.setContentType("application/json")
.setBody("{\"error\":\"Server down\"}")));

page.navigate("/bookings");
assertThat(page.getByText("Something went
wrong")).isVisible();
```

Lesson: Mixing UI and API in one test

src/test/resources/junit-platform.properties

```
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default =
same_thread
junit.jupiter.execution.parallel.mode.classes.def
ault = concurrent
junit.jupiter.execution.parallel.config.strategy
= fixed
junit.jupiter.execution.parallel.config.fixed.par
allelism = 4
```

Lesson: Parallel runs, thread safety and CI

Run it

```
mvn test -Dtest=EssentialsTest -
DbaseUrl=http://localhost:3001
BASE_URL=https://qa.booker.example
API_PASSWORD="$QA_PASSWORD" mvn test
```

Lesson: Real project essentials: environments, secrets, test data, logging and cleanup

TABLES

Table: reason · what it gives you

Reason	What it gives you
One tool for UI and API	One dependency, one set of reports and one CI job
Shared login state	Log in with an API call, then open the browser already signed in
Fast test data	Create bookings with POST before a UI test instead of clicking forms
Network mocking in UI tests	`page.route()` can answer API calls made by the web page
Built in assertions	`assertThat(response).isOK()` prints the response on failure

Lesson: Why test APIs with Playwright

Table: need · option

Need	Option
Query string	`setQueryParam(name, value)`
Header	`setHeader(name, value)`
JSON body	`setData(object or String)`
Form body	`setForm(FormData.create().set("a", "1"))`
File upload	`setMultipart(FormData.create().set("file", path))`
Time limit	`setTimeout(ms)`
Retry on network errors	`setMaxRetries(3)`
Throw on 4xx and 5xx	`setFailOnStatusCode(true)`

Lesson: Building requests with RequestOptions

Table: context option · real situation

Context option	Real situation
`setBaseURL`	local, QA and staging from one suite
`setExtraHTTPHeaders`	a run id so backend logs can be filtered by CI run
`setIgnoreHTTPSErrors`	QA servers with self signed certificates
`setHttpCredentials`	staging behind Basic auth
`setProxy`	corporate networks that force a proxy
`setClientCertificates`	partner and banking APIs with mutual TLS
`setStorageStatePath`	reuse cookies from a login step instead of logging in for every class

Lesson: Real project essentials: environments, secrets, test data, logging and cleanup

Table: feature · since · how it helps an api tester

Feature	Since	How it helps an API tester
Java 25 LTS	September 2025	The new long term support release. Compact source files and instance `main` methods make small API scripts much shorter
Records and pattern matching for `switch`	Java 21	Model responses as records and branch on status codes cleanly
JUnit 6	September 2025	One version number for all JUnit modules, Java 17 as the minimum, and nullness annotations on the API
`@ParameterizedClass`	JUnit 5.13 and 6	Run a whole test class once per environment or per user role
Playwright for Java 1.64	2026	Keep the Java bindings in step with the Node, Python and .NET releases your UI team uses

Lesson: What is new in Java, JUnit 6 and Playwright, and how it helps at work

TIPS AND TRAPS

IN THE EXAM

If your team runs Playwright for UI, this is the natural first step into API testing. Every UI test can start with API calls that create exactly the data it needs.

Lesson: Why test APIs with Playwright

REMEMBER

API tests do not need browsers. Set the environment variable `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1` on CI machines that only run API tests, so the first run does not download Chromium, Firefox and WebKit.

Lesson: Project setup with Maven and JUnit

COMMON MISTAKE Restful Booker answers 403 instead of 401 when auth is missing. Playwright's `setHttpCredentials` waits for a 401 before sending the password, unless you add `.setSend(HttpCredentialsSend.ALWAYS)`. Sending the `Authorization` header yourself, as above, always works.

Lesson: Create, update and delete (CRUD)

REMEMBER When `isOK()` fails, Playwright prints the method, URL, status and the response body. That message is usually enough to understand a failure in CI without rerunning.

Lesson: Assertions and reading JSON

IN THE EXAM Boundary and invalid data rows are where most API bugs hide: empty strings, very long strings, negative prices, a checkout before the checkin, and dates in the wrong format.

Lesson: Data driven tests with @ParameterizedTest

REMEMBER The second test proves the schema itself works. Testing your checks against bad data is a good habit, because a check that can never fail is worth nothing.

Lesson: JSON Schema validation

IN THE EXAM Mocking with `page.route` lets you test how the UI handles slow answers, empty lists and server errors, which are hard to produce on a real server.

Lesson: Mixing UI and API in one test

REMEMBER Tag tests with `@Tag("smoke")` and run a quick subset with `mvn test -Dgroups=smoke` on every commit. Run the full suite nightly.

Lesson: Parallel runs, thread safety and CI

REMEMBER The lambda form `assertEquals(expected, actual, () -> message)` builds the message only when the check fails, so reading the body costs nothing on passing tests.

Lesson: Real project essentials: environments, secrets, test data, logging and cleanup

1.3 Java · HttpClient

The 17 lessons of the Java · HttpClient track.

KEY IDEAS

- **Why learn the native way** An API test sends an HTTP request and checks the response. Libraries make this shorter, but each one sits on top of an HTTP client like this one.
- **Setup: from one file to a Maven project** Java can run a single source file directly, with no build tool. This is the fastest way to try the client. The script below calls the API and exits with code 1 if a check fails, so CI treats it as a failure.
- **Building requests: query strings, headers, methods** HttpClient does not build query strings for you. Encode each value with URLEncoder , or names with spaces and symbols will break the URL.
- **Create, update and delete (CRUD)** Bodies are sent with a BodyPublisher . Text blocks keep JSON readable. Remember to set Content-Type , because the client does not guess it.
- **Reading JSON and asserting well** Parse the body once into a JsonNode with Jackson. Read nested values with JSON Pointer paths. Turn the body into a record when you want to compare whole objects.
- **Authentication and cookies** A CookieManager on the client stores cookies the server sets with Set-Cookie and sends them back on later calls, like a browser. You can also add one yourself.
- **Async calls and parallel requests** sendAsync returns at once with a CompletableFuture . Start many calls together and wait for all of them. This is how you check that an API behaves under a little concurrency.
- **A stub server with the JDK alone** The JDK includes a small HTTP server in com.sun.net.httpserver . Use it as a stub for a service your code depends on, or to test your own client code against answers you control, including errors and slow responses.

- **Schema checks and a small framework** Wrap the raw client in a small class so tests stay short. Add schema validation with networknt, which reads the schema and the body as plain strings.
- **A quick load smoke with virtual threads** This is not a replacement for a load testing tool, but a quick check that an endpoint stays fast under a small burst. Send 50 calls at once, collect the times, and check the 95th percentile.
- **Real project essentials: environments, secrets, logging with redaction and cleanup** Even with no libraries, a native suite can behave like a professional one. The JDK already has everything needed: system properties and environment variables for config, System.Logger for logs, UUID for unique data, and ...
- **Real world scenario: a release gate health check** Before a release goes live, the team wants a quick yes or no: is the booking service healthy and does a full booking journey still work? The check must run anywhere a JDK exists, finish in seconds, and print a clear ...

FORMULAS

A small query helper

```
          import java.net.URLEncoder;
    import java.nio.charset.StandardCharsets;
             import java.util.Map;
       import java.util.stream.Collectors;

static String query(Map<String, String> params) {
          return params.entrySet().stream()
         .map(e -> URLEncoder.encode(e.getKey(),
         StandardCharsets.UTF_8) + "="
               + URLEncoder.encode(e.getValue(),
           StandardCharsets.UTF_8))
         .collect(Collectors.joining("&", "?",
                     ""));
                       }

    URI uri = URI.create(BASE + "/booking" +
    query(Map.of("firstname", "Mary Ann")));
// https://restful-booker.herokuapp.com/booking?
               firstname=Mary+Ann
```

Lesson: Building requests: query strings, headers, methods

Cookie manager

```
  CookieManager cookies = new CookieManager();
               HttpClient client =
HttpClient.newBuilder().cookieHandler(cookies).bu
                     ild();

 HttpCookie c = new HttpCookie("token", token);
                c.setPath("/");
cookies.getCookieStore().add(URI.create(BASE),
   c);   // now sent with every call to BASE
```

Lesson: Authentication and cookies

Run it

```
        mvn test -Dtest=EssentialsTest -
         DbaseUrl=http://localhost:3001
          API_PASSWORD="$QA_PASSWORD"
  BASE_URL=https://qa.booker.example mvn test
```

Lesson: Real project essentials: environments, secrets, logging with redaction and cleanup

TABLES

Table: class · job

Class	Job
`HttpClient`	Sends requests. Holds settings such as timeouts, redirects, cookies and HTTP version. Create one and reuse it.
`HttpRequest`	One request: URI, method, headers, body. Immutable, built with a builder.
`HttpRequest.BodyPublishers`	Turns a String, bytes or file into a request body.
`HttpResponse`	Status code, headers, body and the request that produced it.
`HttpResponse.BodyHandlers`	Reads the body as a String, bytes, file, stream or nothing.

Lesson: Why learn the native way

Table: http method · builder call

HTTP method	Builder call
GET	`.GET()`, the default
POST	`.POST(BodyPublishers.ofString(json))`
PUT	`.PUT(BodyPublishers.ofString(json))`
PATCH	`.method("PATCH", BodyPublishers.ofString(json))`
DELETE	`.DELETE()`
HEAD	`.HEAD()`

Lesson: Building requests: query strings, headers, methods

Table: feature · since · how it helps an api tester

Feature	Since	How it helps an API tester
Java 25 LTS	September 2025	The current long term support release, a good target for new CI images
Compact source files and instance `main`	Java 25 (JEP 512)	A health check or data seeding script in a few lines, run with `java Seed.java`
Module import declarations	Java 25 (JEP 511)	`import module java.net.http;` brings in the whole HTTP client in one line
Scoped values	Java 25 (JEP 506)	Pass a request id or test run id safely through virtual threads without thread locals
Virtual threads	Java 21	Thousands of concurrent requests for a quick load smoke, as in the earlier lesson
HTTP/3 in `HttpClient`	Java 26 (JEP 517, March 2026)	Opt in with `HttpClient.Version.HTTP_3` to test gateways that speak HTTP/3 over QUIC. HTTP/2 has been there since Java 11

Lesson: What is new in Java for HTTP testing, and how it helps at work

TIPS AND TRAPS

IN THE EXAM Many companies have internal tools that check service health before a release. A single Java file using HttpClient can do that with nothing to install except the JDK.

Lesson: Why learn the native way

COMMON MISTAKE Some header names are restricted and cannot be set, such as `Host`, `Connection` and `Content-Length`. The client sets them itself.

Lesson: Building requests: query strings, headers, methods

REMEMBER `BodyHandlers.discarding()` skips reading the body when you only need the status, which saves memory on large responses.

Lesson: Create, update and delete (CRUD)

REMEMBER `path("x")` returns a missing node when the field is absent, while `get("x")` returns null. Use `path` when a field is optional to avoid a `NullPointerException`.

Lesson: Reading JSON and asserting well

COMMON MISTAKE The JDK `Authenticator` only answers a 401 challenge. Restful Booker answers 403, so build the `Authorization` header yourself as shown.

Lesson: Authentication and cookies

REMEMBER In Java 21 you can also start one virtual thread per call with `Executors.newVirtualThreadPerTaskExecutor()` and use the simple blocking `send` inside each. The code reads top to bottom and still runs in parallel.

Lesson: Async calls and parallel requests

IN THE EXAM A table of path and expected status is a quick way to cover routing, missing ids and wrong id types in one place. Add rows whenever a bug is found so it never returns.

Lesson: Data driven tests

REMEMBER The timeout test proves your code gives up on a slow service instead of hanging forever. Production outages often start with a missing timeout.

Lesson: A stub server with the JDK alone

COMMON MISTAKE Never run load against a shared public API such as Restful Booker beyond a small burst. Use your own test environment for real load testing with tools such as Gatling, k6 or JMeter.

Lesson: A quick load smoke with virtual threads

COMMON MISTAKE The regular expressions here keep the example free of libraries. In a real suite, parse JSON with Jackson as the schema lesson does. Regex parsing breaks as soon as the response format changes.

Lesson: Real project essentials: environments, secrets, logging with redaction and cleanup

REMEMBER Before you move a suite to a new Java release at work, run it on the new JDK in a separate CI job for a week. Compile warnings about deprecated APIs are your to do list.

Lesson: What is new in Java for HTTP testing, and how it helps at work

STEP 2 Python

2.1 Python · requests and pytest

The 19 lessons of the Python · requests and pytest track.

KEY IDEAS

- **What API automation is and what to check** An API test sends a real HTTP request and checks the response, with no browser. API tests are fast and stable, so teams keep many of them in the middle of the test pyramid.
- **Fixtures and conftest.py** A fixture is a function that prepares something a test needs. A test asks for it by naming it as an argument. Code after yield runs after the test, which makes cleanup automatic.
- **Schema checks with jsonschema and pydantic** Two good options. jsonschema checks a response against a JSON Schema file, which can be shared with other teams and tools.
- **Framework design: a client class** Keep URLs and HTTP details in one client class. Tests call methods with clear names and only contain checks. When an endpoint changes, you fix it in one place.
- **Mocking HTTP with responses and respx** When you test code that calls an API, such as your own client class, you often want to control the answers. responses intercepts calls made with requests. respx does the same for httpx.
- **Async tests with httpx** httpx has the same style as requests, plus an async client. With asyncio.gather you can send many calls at once. This is useful for quick concurrency checks and for testing async services.
- **Retries, polling and flaky networks** Real networks drop calls now and then. Retry the safe calls at the HTTP layer so a single blip does not fail the build, and poll properly when an API finishes work in the background.
- **Expert topics: property based testing, contracts and load** Schemathesis reads an OpenAPI spec and generates thousands of requests, including odd values you would not think of. It checks that the API never answers 500 and that every response matches the spec.
- **Real project essentials: environments, secrets, fake data, logging and cleanup** Tutorials talk to one server with one password. Real teams run the same suite against local, QA and staging, keep passwords out of Git, need fresh realistic data for every run, and want to see the exact request and ...
- **Real world scenario: a hotel front desk day** A receptionist books a room for a guest, adds breakfast later, and then the guest cancels. Someone without a login also tries to cancel the booking. The test proves each step, using fixtures for login.

FORMULAS

Terminal

```
python -m venv .venv
source .venv/bin/activate          # Windows:
.venv\Scripts\activate
pip install -r requirements.txt
pytest                             # run
everything
pytest -k crud -v                  # run tests
whose name contains "crud"
BASE_URL=http://localhost:3001 pytest   # point
at another server
```

Lesson: Project setup: venv, packages and layout

tests/status_cases.json

```
[
  { "path": "/ping",           "status": 201 },
  { "path": "/booking",        "status": 200 },
  { "path": "/booking/1",      "status": 200 },
  { "path": "/booking/999999", "status": 404 },
  { "path": "/booking/abc",    "status": 404 }
]
```

Lesson: Data driven tests with parametrize

Common commands

```
pytest -m smoke                        # only
tests marked smoke
pytest -n auto                         # parallel
on every CPU core (pytest-xdist)
pytest -n 4 --dist loadfile            # 4
workers, tests from one file stay together
pytest --junitxml=reports/junit.xml    # report CI
systems understand
pytest --lf                            # rerun
only the last failures
```

Lesson: Markers, parallel runs, reports and CI

Automatic retries on a session

```
from requests.adapters import HTTPAdapter
from urllib3.util import Retry

retry = Retry(
total=3,
backoff_factor=0.5,                    #
waits 0.5 s, 1 s, 2 s
status_forcelist=[502, 503, 504],
allowed_methods=["GET", "PUT", "DELETE"], #
never retry POST automatically
)
session.mount("https://",
HTTPAdapter(max_retries=retry))
```

Lesson: Retries, polling and flaky networks

One command against an OpenAPI spec

```
pip install schemathesis
st run https://api.example.com/openapi.json --
checks all
```

Lesson: Expert topics: property based testing, contracts and load

pyproject.toml (pytest 9 native TOML mode)

```
[tool.pytest]
testpaths = ["tests"]
pythonpath = ["."]
addopts = ["-ra"]
strict = true
markers = [
"smoke: quick checks that run on every
commit",
]
```

Lesson: What is new in pytest 9 and Python, and how it helps at work

TABLES

Table: check · example · python

Check	Example	Python
Status	GET /booking/1 gives 200	`assert res.status_code == 200`
Body value	firstname is a string	`assert isinstance(res.json()["firstname"], str)`
Header	JSON content type	`assert "application/json" in res.headers["Content-Type"]`
Time	under 2 seconds	`assert res.elapsed.total_seconds() < 2`
Negative	update without token gives 403	`assert res.status_code == 403`
Shape	matches the schema	`validate(res.json(), schema)`

Lesson: What API automation is and what to check

Table: argument · sends

Argument	Sends
`params=`	query string
`json=`	a JSON body and the `Content-Type: application/json` header
`data=`	form fields or raw text
`files=`	multipart file upload
`headers=`	extra headers
`cookies=`	cookies
`auth=`	Basic auth as a tuple
`timeout=`	seconds, or a (connect, read) tuple

Lesson: Building requests: params, headers, cookies and sessions

Table: scope · created · good for

Scope	Created	Good for
`function` (default)	once per test	test data that a test changes
`module`	once per file	data shared by a file of read only tests
`session`	once per run	the HTTP session, config, auth token

Lesson: Fixtures and conftest.py

Table: habit · why · here

Habit	Why	Here
Config from the environment	One suite, many servers, no code changes	`TEST_ENV` and `BASE_URL`
Secrets never in Git	Leaked passwords are a security incident	Defaults only for the public practice API
Fake but realistic data	Unique names avoid clashes between parallel runs and catch encoding bugs	`Faker("en_IN")`
Log every exchange	A failing CI test explains itself	a `requests` response hook
Clean up what you create	Shared QA databases stay usable	a fixture with code after `yield`

Lesson: Real project essentials: environments, secrets, fake data, logging and cleanup

Table: feature · since · how it helps an api tester

Feature	Since	How it helps an API tester
Built in `subtests` fixture	pytest 9.0	Check many fields of one response and see every failure, not only the first
Native TOML config in `[tool.pytest]`	pytest 9.0	Real lists, numbers and booleans in `pyproject.toml` instead of strings
`strict = true`	pytest 9.0	One switch for strict markers, strict config and strict xfail, so typos in marker names fail the run
Python 3.14	October 2025	Template strings (t-strings, PEP 750) and the officially supported free threaded build for heavier parallel scripts
requests 2.34 and httpx 0.28	2025 and 2026	Current TLS defaults and security fixes. Keep them pinned and upgrade on purpose

Lesson: What is new in pytest 9 and Python, and how it helps at work

TIPS AND TRAPS

IN THE EXAM — pytest prints the actual values when an `assert` fails, for example `assert 404 == 200`. You do not need special assertion methods, which keeps tests easy to read for the whole team.
Lesson: What API automation is and what to check

REMEMBER — pytest finds files named `test_*.py` and functions named `test_*`. No classes or main method are needed.
Lesson: Project setup: venv, packages and layout

REMEMBER — Use a `Session` in test suites. It reuses connections, so a suite of hundreds of calls runs noticeably faster.
Lesson: Building requests: params, headers, cookies and sessions

REMEMBER — Comparing whole dicts with == is one of Python's best features for API tests. When they differ, pytest shows exactly which keys changed.
Lesson: Create, update and delete (CRUD)

REMEMBER — Every test gets its own booking and cleans it up, even when the test fails. Tests never depend on each other or on data someone else created.
Lesson: Fixtures and conftest.py

IN THE EXAM — Restful Booker returns 200 with `{"reason": "Bad credentials"}` for a wrong password. Many teams would log this as a bug, because 401 is the correct status. Your test documents the real behaviour and makes it visible.
Lesson: Authentication styles

COMMON MISTAKE — Keep secrets in environment variables or a CI secret store. Never print tokens in test output.
Lesson: Authentication styles

REMEMBER — The `ids` list gives each row a readable name in the report, such as `test_names_are_kept[accents]`.
Lesson: Data driven tests with parametrize

REMEMBER — pydantic can check business rules too, such as checkout after checkin, which a plain schema cannot express. Use `extra="forbid"` to fail when the API adds fields you did not expect.
Lesson: Schema checks with jsonschema and pydantic

REMEMBER — Run with `pytest --log-cli-level=INFO` to see the line logged for every call by the response hook.
Lesson: Framework design: a client class

IN THE EXAM — Mocks test your code, not the real API. Use them for unit tests of helpers and clients, and keep separate tests that call the real service.
Lesson: Mocking HTTP with responses and respx

REMEMBER — Bugs such as duplicate ids or lost updates only appear when calls overlap. A few parallel calls in a test can expose them early.
Lesson: Async tests with httpx

COMMON MISTAKE — Retries hide real problems if overused. Log every retry, keep the count low, and never retry a 4xx, since the request itself is wrong.
Lesson: Retries, polling and flaky networks

REMEMBER pytest shows captured log lines only for failing tests, so passing runs stay quiet and failing runs show the full request and response. That is exactly what you want in CI.

Lesson: Real project essentials: environments, secrets, fake data, logging and cleanup

2.2 Python · Playwright

The 19 lessons of the Python · Playwright track.

KEY IDEAS

- **Why test APIs with Playwright for Python** An API test sends an HTTP request and checks the response, with no browser. Playwright only starts a browser when a test asks for a page , so pure API tests run as fast as tests written with requests.
- **Assertions, readable failures and polling** Playwright for Python has one web first assertion for API responses, expect(res).to_be_ok() . Everything else is a plain pytest assert , which prints both sides when it fails.
- **Fixtures for tokens and test data** Fixtures prepare what a test needs and clean it up afterwards. The code after yield is the cleanup, and pytest runs it even when the test fails. A session scoped fixture runs once per run, which suits a login token.
- **Framework design: a client class and fixtures** As a suite grows, keep URLs and headers in one class with one method per endpoint. Tests then read like business steps, and a changed endpoint is fixed in one place.
- **UI and API together** This is where Playwright beats every other Python HTTP client. Inside a UI test, page.request shares cookies with the browser, page.expect_response captures what the page sends, and page.route fakes API answers.
- **Async API calls for speed** The sync API sends one call at a time. The async API in playwright.async_api can keep many calls in flight with asyncio.gather . Use it for seeding lots of data, quick load checks and concurrency tests.
- **Real project essentials: environments, secrets, fake data, logging and cleanup** A company suite runs against several servers, keeps passwords out of Git, creates fresh data for every run, explains its own failures, and leaves the shared QA database tidy.
- **Real world scenario: a hotel front desk day** The receptionist logs in, books a room, adds breakfast, and the guest later cancels. Someone without a login also tries to cancel. Each step is a small function, so a failure names the exact step that broke.

FORMULAS

Terminal

```
              python -m venv .venv
source .venv/bin/activate              # Windows:
             .venv\Scripts\activate
         pip install -r requirements.txt
  pytest                                  # run
                    everything
  pytest -m smoke                         # only
               tests marked smoke
  pytest --base-url http://localhost:3001     #
             point at another server
 pytest --tracing retain-on-failure     # keep a
           trace for every failed test
```

Lesson: Project setup: packages, pytest.ini and conftest.py

api/__init__.py

```
from api.booking_client import BookingClient

          __all__ = ["BookingClient"]
```

Lesson: Framework design: a client class and fixtures

Log in once by API and reuse the session in every UI test

```
        @pytest.fixture(scope="session")
    def storage_state(playwright, base_url,
                      tmp_path_factory):
      path = tmp_path_factory.mktemp("auth") /
                     "qa.json"
                         ctx =
playwright.request.new_context(base_url=base_url)
     ctx.post("/api/login", form={"user": "qa",
    "password": os.environ["QA_PASSWORD"]})
    ctx.storage_state(path=path)              #
        saves the cookies the login set
                     ctx.dispose()
                      return path


                  @pytest.fixture
 def browser_context_args(browser_context_args,
                  storage_state):
          return {**browser_context_args,
        "storage_state": storage_state}
```

Lesson: UI and API together

Run options

```
pytest -n 4                                  # 4
       worker processes with pytest-xdist
 pytest -m "smoke and not slow"               #
             pick tests by marker
 pytest --lf                                  #
           rerun only last failures
 pytest --tracing retain-on-failure           #
         trace.zip for each failed test
   playwright show-trace test-results/<test-
                name>/trace.zip
 pytest --junitxml=results/junit.xml          #
           report for CI dashboards
```

Lesson: Markers, parallel runs, traces and CI

TABLES

Table: feature · why testers like it

Feature	Why testers like it
`APIRequestContext`	base URL, default headers, cookies and timeouts set once for every call
`expect(res).to_be_ok()`	prints the status and the response body when the status is not 2xx
Traces	every call with its headers, body and timing, viewable after a CI failure
Shared cookies with the browser	`page.request` calls the API as the signed in user
Network mocking	`page.route` fakes API answers so the UI can be tested against errors
Sync and async APIs	plain functions for pytest, or `async` code for many calls at once

Lesson: Why test APIs with Playwright for Python

Table: question · requests with pytest · playwright with python

Question	requests with pytest	Playwright with Python
Only API tests, no UI at all	Simplest choice	Works well too
UI tests already use Playwright	Second client to learn	Same project and fixtures
Trace of every call in CI	Write your own logging	Built in
Seed data before a UI test	Separate session, separate cookies	`page.request` shares the login

Lesson: Why test APIs with Playwright for Python

Table: read · gives

Read	Gives
`res.status`	the status code as an int, for example 200
`res.ok`	`True` for any 2xx status
`res.headers`	a dict with lower case header names
`res.json()`	the body parsed into dicts and lists
`res.text()`	the body as a string, useful for plain text answers such as 404 Not Found
`res.body()`	the raw bytes, for files and images

Lesson: Your first test: GET a booking

Table: option · sends

Option	Sends
`params=`	query string, encoded for you
`data=`	JSON for dicts and lists, raw text for strings, raw bytes for bytes
`form=`	application/x-www-form-urlencoded
`multipart=`	multipart/form-data, for files
`headers=`	extra headers for this call, added to the context headers
`timeout=`	time limit in milliseconds, 0 means no limit
`fail_on_status_code=`	raise instead of returning a 4xx or 5xx response
`max_retries=`	retries for network errors only, never for bad status codes

Lesson: Request options: params, data, form, headers and timeouts

Table: scope · runs · good for

Scope	Runs	Good for
`function` (default)	once per test	test data that a test changes
`module`	once per file	read only data shared by a file
`session`	once per run, or once per xdist worker	the API client and the login token

Lesson: Fixtures for tokens and test data

Table: layer · holds · never holds

Layer	Holds	Never holds
`api/` client	paths, headers, auth, request building	assertions
`conftest.py`	clients, tokens, test data, cleanup	business checks
`tests/`	steps and assertions	URLs or raw headers

Lesson: Framework design: a client class and fixtures

Table: new_context option · real situation it solves

new_context option	Real situation it solves
`base_url`	the same tests against local, QA and staging
`extra_http_headers`	a run id header so backend logs can be filtered by test run
`ignore_https_errors`	QA servers with self signed certificates (never for production checks)
`http_credentials`	staging sites behind Basic auth
`proxy`	corporate networks that only allow traffic through a proxy
`client_certificates`	banking and partner APIs that need mutual TLS
`storage_state`	reuse cookies saved by a login step, so every test does not log in again
`fail_on_status_code`	turn every 4xx and 5xx into an exception, handy for setup calls

Lesson: Real project essentials: environments, secrets, fake data, logging and cleanup

Table: feature · available in · how it helps an api tester

Feature	Available in	How it helps an API tester
Tracing on an API request context	Playwright 1.64	Record every call of a pure API test into a trace file and open it in the trace viewer, with no browser
Cookie methods on API contexts	Playwright 1.64	`add_cookies`, `cookies` and `clear_cookies` to set or inspect a session without a browser
`timedelta` timeouts	Playwright 1.64 for Python	`timeout=timedelta(seconds=10)` reads better than a number of milliseconds
`max_retries` per call	Playwright 1.46 and newer	retry connection resets on flaky test networks without retrying real 5xx bugs
Built in `subtests`	pytest 9.0	check many fields in one test and see every failure
Native TOML config	pytest 9.0	`[tool.pytest]` in `pyproject.toml` with real lists and booleans

Lesson: What is new in Playwright and pytest, and how it helps at work

TIPS AND TRAPS

IN THE EXAM — If your team already writes Playwright UI tests in Python, add API tests to the same repo. Start by replacing slow UI setup steps, such as creating a booking through five screens, with a single API call.

Lesson: Why test APIs with Playwright for Python

REMEMBER — API only projects never need a browser. Skip `playwright install` until you add UI tests, which keeps CI images small.

Lesson: Project setup: packages, pytest.ini and conftest.py

COMMON MISTAKE — In Python, `status`, `ok`, `headers` and `url` are properties, while `json()`, `text()` and `body()` are methods. Writing `res.status()` raises a TypeError.

Lesson: Your first test: GET a booking

REMEMBER — Timeouts in Playwright are milliseconds. In requests and httpx they are seconds. Mixing them up gives a 10 millisecond timeout that fails every call.

Lesson: Request options: params, data, form, headers and timeouts

COMMON MISTAKE — One long test like this stops at the first failed step and leaves the booking behind. The fixtures lesson shows how to give every test its own booking that is always cleaned up.

Lesson: Create, update and delete (CRUD)

REMEMBER Add a message to important asserts, for example `assert res.status == 200, res.text()`. When the test fails in CI, the message shows the error body the server sent.

Lesson: Assertions, readable failures and polling

REMEMBER Every test gets its own booking and removes it afterwards. Tests can run in any order and in parallel without breaking each other.

Lesson: Fixtures for tokens and test data

COMMON MISTAKE Without `"send": "always"`, `http_credentials` waits for a 401 challenge before sending the password. Restful Booker answers 403, so the password would never be sent and the test would fail.

Lesson: Authentication and negative tests

IN THE EXAM For every protected endpoint, test at least three cases: no credentials, wrong credentials and a valid token that lacks the right role. Then read the data again to prove nothing changed.

Lesson: Authentication and negative tests

REMEMBER Give each case a readable id with `pytest.param(..., id="...")` or `ids=`. The report then says `test_update_is_refused[wrong password]` instead of a list of numbers.

Lesson: Data driven tests with parametrize

REMEMBER The second test in each pair checks the checker. A schema that never fails protects nothing.

Lesson: Schema checks with jsonschema and pydantic

IN THE EXAM `expect_response` lets a UI test assert on the exact request the page sent. It catches bugs where the screen looks right but the wrong data reaches the server.

Lesson: UI and API together

COMMON MISTAKE Do not share one sync Playwright object between threads. For parallel work use the async API, or run tests in separate processes with pytest-xdist.

Lesson: Async API calls for speed

REMEMBER No `playwright install` step is needed for API tests. Add `playwright install --with-deps chromium` only when the suite gains UI tests.

Lesson: Markers, parallel runs, traces and CI

COMMON MISTAKE Faker data is random, so a failure might not repeat. Print the data in the log, or seed Faker with `Faker.seed(1234)` in a run you need to reproduce.

Lesson: Real project essentials: environments, secrets, fake data, logging and cleanup

REMEMBER Keep a trace for failing CI tests only. Start tracing in a fixture, and save the file when `request.node` reports a failure, so CI artifacts stay small.

Lesson: What is new in Playwright and pytest, and how it helps at work

IN THE EXAM Real front desk systems add more steps, such as payment and email. The pattern stays the same: create your own data, act through the API, check the state after every step, and clean up.

Lesson: Real world scenario: a hotel front desk day

STEP 3 JavaScript

3.1 JavaScript · Postman and Newman

The 17 lessons of the JavaScript · Postman and Newman track.

KEY IDEAS

- **From manual requests to automated tests** An API test sends a request and checks the response. In Postman you already send requests by hand.
- **Your first request with a test** Create a request GET {{baseUrl}}/ping . Open Scripts , then Post-response , and add the script below. Press Send, then open Test Results under the response.
- **Variables and their scopes** Write {{name}} anywhere in a URL, header or body and Postman replaces it before sending. In scripts you read and write variables with pm .
- **CRUD requests and chaining with variables** Each request saves what the next one needs. The login stores token , the create stores bookingId , and later requests use {{token}} and {{bookingId}} .
- **Assertions cookbook** pm.expect is the Chai assertion library. Chains read like sentences and give clear failure messages.
- **Authorization tab and auth scripts** Set auth once on the collection or a folder, then choose Inherit auth from parent on each request. Postman supports the common types directly.
- **Data driven runs with CSV or JSON** Run the same requests once per row of a data file. Column names become variables, so {{firstname}} in the body is filled from the row. In scripts, read the row with pm.iterationData .
- **JSON Schema checks** pm.response.to.have.jsonSchema(schema) validates the body with Ajv, which is built into the Postman sandbox. One schema check replaces many field checks and catches renamed or retyped fields.
- **Controlling the run: setNextRequest, skips and loops** By default the runner plays requests from top to bottom. Scripts can change that.
- **Newman: reporters, options and running from Node** Newman is also a Node library, so you can run collections from a script and decide what to do with the results.
- **Real project essentials: environments, secrets, safe logging and cleanup** When a whole team shares one workspace, a few habits keep it safe: one environment per server, secrets that never sync to the cloud or Git, a guard that stops destructive tests on production, logs that never print ...
- **Real world scenario: the front desk collection** A receptionist logs in, books a room, the guest adds breakfast, someone without a login tries to cancel, and finally the booking is cancelled. Here is the full collection, built from the scripts in the earlier lessons.

FORMULAS

Terminal

```
npm install -g newman
newman --version

# export the collection from Postman (... >
Export > v2.1), then:
newman run booker.postman_collection.json
newman run booker.postman_collection.json -e
local.postman_environment.json
newman run booker.postman_collection.json --env-
var baseUrl=http://localhost:3001
```

Lesson: Setup: workspace, environment and Newman

Ping: Post-response script

```
pm.test("API is up", () ⇒ {
pm.response.to.have.status(201);
});
```

Lesson: Your first request with a test

POST {{baseUrl}}/auth: Post-response

```
pm.test("Login gives a token", () ⇒ {
pm.response.to.have.status(200);
const body = pm.response.json();
pm.expect(body.token,
"token").to.be.a("string").and.not.be.empty;
pm.collectionVariables.set("token",
body.token);
});
```

Lesson: CRUD requests and chaining with variables

bookings.csv

```
firstname,lastname,totalprice,expected
Ann,Lee,100,200
José,Núñez,250,200
O'Brien,Smith-Jones,0,200
```

Lesson: Data driven runs with CSV or JSON

Collection level Post-response: shared checks for every request

```
// Runs after every request in the collection
pm.test("No server errors", () ⇒
pm.expect(pm.response.code).to.be.below(500));
```

Lesson: Expert habits for large collections

Collection Pre-request script: the production guard

```
// Requests whose name starts with [destructive]
never run against production
const env = pm.environment.name ||
pm.variables.get("envName") || "";
if (/prod/i.test(env) &&
pm.info.requestName.startsWith("[destructive]"))
{
console.warn(`Skipping
"${pm.info.requestName}" on ${env}`);
pm.execution.skipRequest();
}
```

Lesson: Real project essentials: environments, secrets, safe logging and cleanup

Postman CLI in a pipeline

```
npm install -g postman-cli #
or the installer from Postman's site
postman login --with-api-key "$POSTMAN_API_KEY"
postman collection run "$COLLECTION_ID" -e
"$ENVIRONMENT_ID" --reporters cli,junit
```

Lesson: What is new in Postman, and how it helps at work

TABLES

Table: postman part · what it is

Postman part	What it is
Collection	A folder of saved requests that run in order. This is your test suite.
Environment	A set of variables such as `baseUrl` for dev, QA or staging.
Pre-request script	JavaScript that runs before the request, for example to build test data.
Post-response script	JavaScript that runs after, with `pm.test` checks. Older versions call this the Tests tab.
Collection Runner	Runs a collection inside the app, optionally with a data file.
Newman and Postman CLI	Run a collection from the terminal or CI.

Lesson: From manual requests to automated tests

Table: scope · lives in · read and write in scripts · use for

Scope	Lives in	Read and write in scripts	Use for
Global	the workspace	`pm.globals`	rarely, values for every collection
Collection	one collection	`pm.collectionVariables`	ids and tokens passed between requests
Environment	the selected environment	`pm.environment`	base URL and users per server
Data	the current data file row	`pm.iterationData`	data driven runs
Local	one request run	`pm.variables`	temporary values

Lesson: Variables and their scopes

Table: request · expected

Request	Expected
`PATCH /booking/{{bookingId}}` with the token	200 and the changed field
`DELETE /booking/{{bookingId}}` without the token	403
`DELETE /booking/{{bookingId}}` with the token	201, a quirk of this API
`GET /booking/{{bookingId}}`	404

Lesson: CRUD requests and chaining with variables

Table: auth type · fill in · sends

Auth type	Fill in	Sends
Basic Auth	`admin` and `{{password}}`	`Authorization: Basic ...` on every request
Bearer Token	`{{accessToken}}`	`Authorization: Bearer ...`
API Key	key name, value, header or query	for example `x-api-key`
OAuth 2.0	token URL, client id and secret, scope	gets and refreshes tokens for you

Lesson: Authorization tab and auth scripts

Table: habit · how to do it in postman

Habit	How to do it in Postman
One environment per server	`Local`, `QA`, `Staging`, each with the same variable names such as `baseUrl`
Secrets out of sync and Git	Mark variables as **secret** type, keep values in the **current value** only, or store them in **Postman Vault** and use `{{vault:qa-password}}`
Secrets in CI	Pass them at run time: `newman run ... --env-var "password=$QA_PASSWORD"`
Guard production	A collection level pre-request script that skips risky requests when the environment is production
Safe logs	Log status and timing to the Postman Console, never the token
Cleanup	A last folder named `Teardown` that deletes what the run created

Lesson: Real project essentials: environments, secrets, safe logging and cleanup

Table: feature · what it is · how it helps an api tester

Feature	What it is	How it helps an API tester
Postman 12, Git native	Collections, specs, flows and local mocks live in your Git repository, on the same branch as the code	An API change and its tests arrive in one pull request and are reviewed together
Collection 3.0 format	YAML instead of one big JSON file, split into one file per request	Small, readable diffs in code review, and easy for AI tools to read and edit
Code based local mock servers	Mocks that run on your laptop or in CI, versioned with the code	Test the frontend or a consumer before the real backend is ready
Multi protocol collections	HTTP, GraphQL, gRPC, WebSocket, MQTT, MCP and AI requests in one collection	One test plan for a whole system, including MCP servers used by AI agents
Agent Mode	An AI agent inside Postman that creates and edits collections, tests and mocks, and diagnoses failed tests	Faster first drafts of tests. You still review every assertion it writes
Postman CLI	Postman's own command line runner, `postman collection run`	Runs collections from your workspace in CI and reports results back to Postman
Package Library and `pm.require`	Reusable script packages shared across a team	One tested helper for auth or schema checks instead of copies in every request

Lesson: What is new in Postman, and how it helps at work

TIPS AND TRAPS

IN THE EXAM — This is the friendliest path from manual to automated API testing. You keep the requests you already trust and add checks one at a time.
Lesson: From manual requests to automated tests

REMEMBER — The snippets list on the right of the script editor inserts common checks with one click, such as status code and response time. They are a good way to learn the `pm` API.
Lesson: Your first request with a test

COMMON MISTAKE — Mark secrets such as passwords as **secret** type in the environment, and do not commit exported environments that contain real secrets.
Lesson: Variables and their scopes

REMEMBER — A failed `pm.expect` inside one `pm.test` does not stop the other tests in the script, so one run shows every problem.
Lesson: Assertions cookbook

IN THE EXAM — Data files are a simple way to hand test design to the whole team. Testers add rows for boundary values and special characters without touching any script.
Lesson: Data driven runs with CSV or JSON

REMEMBER — Store large schemas in a collection variable or a Postman package and load them with `JSON.parse(pm.collectionVariables.get("bookingSchema"))`, so many requests share one schema.
Lesson: JSON Schema checks

COMMON MISTAKE — Jumps only work in the Collection Runner, Newman and the Postman CLI, not when you press Send on a single request. Always add a limit to loops.
Lesson: Controlling the run: setNextRequest, skips and loops

REMEMBER — The official **Postman CLI** (`postman collection run`) can run collections stored in your Postman workspace and send results back to Postman. Newman runs exported files and needs no account.
Lesson: Newman: reporters, options and running from Node

COMMON MISTAKE — The **initial value** of a variable syncs to the Postman cloud and is shared with everyone in the workspace. Put real secrets only in the **current value**, in Vault, or in CI secrets.
Lesson: Real project essentials: environments, secrets, safe logging and cleanup

IN THE EXAM — Import this file into Postman with **Import**, then run it in the Collection Runner to see every test result in the app.
Lesson: Real world scenario: the front desk collection

3.2 JavaScript · Playwright

The 18 lessons of the JavaScript · Playwright track.

KEY IDEAS

- **Why test APIs with Playwright Test** An API test sends a request and checks the response, with no browser. Playwright Test does not start a browser unless a test asks for a page , so pure API tests run in milliseconds and in parallel across workers.
- **Custom fixtures for tokens and test data** Fixtures prepare what a test needs and clean it up afterwards. Code after await use(value) is the cleanup, and it runs even when the test fails. A worker fixture runs once per worker process, which suits a login token.
- **Data driven tests** Playwright has no special data annotation. A normal loop that calls test() creates one named test per row, and each row can run in parallel.

- **UI and API together** Inside a UI test, page.request shares cookies with the browser. Three patterns save QA teams the most time.
- **Real project essentials: environments, secrets, test data, logging and cleanup** A company suite runs against several servers, keeps secrets out of Git, makes unique data so parallel workers never clash, attaches the exact exchange to the report when something fails, and leaves the shared QA ...
- **Real world scenario: a hotel front desk day** The receptionist logs in, books a room, adds breakfast, and the guest later cancels. Someone without a login also tries to cancel. Each step is a test.step , so the report reads like the story.

FORMULAS

Terminal

```
                  npm init -y
 npm install -D @playwright/test@1.64.1 ajv ajv-
                    formats
npx playwright test                          # run
                  everything
npx playwright test tests/crud.spec.js      # one
                    file
  npx playwright test --grep @smoke           #
                tagged tests only
  npx playwright show-report                  #
              open the HTML report
  BASE_URL=http://localhost:3001 npx playwright
                    test
```

Lesson: Project setup and config

Group steps so the report reads like a test case

```
await test.step('create a booking', async () => {
                  /* ... */ });
  await test.step('check it appears in search',
           async () => { /* ... */ });
```

Lesson: Assertions: matchers, soft checks and polling

Common styles

```
                  // Per call
  await request.patch(url, { headers: { Cookie:
          `token=${token}` }, data });
      await request.get(url, { headers: {
Authorization: `Bearer ${process.env.API_TOKEN}`
                    } });

// For every call in the project, from the config
use: { extraHTTPHeaders: { Authorization: `Bearer
          ${process.env.API_TOKEN}` } }

  // Basic auth sent up front, for a new context
const api = await playwright.request.newContext({
                    baseURL,
  httpCredentials: { username: 'admin', password:
        'password123', send: 'always' },
                    });
```

Lesson: Authentication and negative tests

Run options

```
npx playwright test --workers 4               #
              parallel processes
npx playwright test --shard 1/3               #
           split across 3 CI machines
npx playwright test --repeat-each 5           #
              hunt for flaky tests
npx playwright test --last-failed             #
              rerun failures only
        npx playwright show-trace test-
              results/**/trace.zip
```

Lesson: Workers, sharding, traces and CI

TABLES

Table: feature · why testers like it

Feature	Why testers like it
`request` fixture	ready to use, with base URL and headers from the config
`expect(res).toBeOK()`	prints the response body when the status is not 2xx
Custom fixtures	tokens and test data created and cleaned up for you
Parallel workers	files run at the same time with no extra setup
Trace viewer and HTML report	every API call with headers and bodies, after the run
UI and API together	seed data, log in and mock the network inside UI tests

Lesson: Why test APIs with Playwright Test

Table: option · sends

Option	Sends
`params`	query string, encoded for you
`data`	JSON for objects, raw for strings and buffers
`form`	application/x-www-form-urlencoded
`multipart`	multipart/form-data, for files
`headers`	extra headers for this call
`timeout`	time limit in ms
`maxRetries`	retries for network errors only

Lesson: Request options

Table: need · playwright answer

Need	Playwright answer
Different servers	`baseURL` chosen from `TEST_ENV` in the config
Secrets	`process.env`, filled from CI secrets, or a local `.env` loaded with `node --env-file` or the dotenv package
Unique data	`crypto.randomUUID()` in the names
Readable failures	`test.info().attach()` and `trace: 'retain-on-failure'`
Cleanup	code after `await use()` in a fixture

Lesson: Real project essentials: environments, secrets, test data, logging and cleanup

Table: feature · available in · how it helps an api tester

Feature	Available in	How it helps an API tester
Tracing on API request contexts	Playwright 1.64	API calls appear in the trace viewer with headers and bodies. The runner traces them for you when `trace` is on
Cookie methods on API contexts	Playwright 1.64	`addCookies()`, `cookies()` and `clearCookies()` set or inspect a session without a browser
`await using` on request contexts	Playwright 1.64 with Node 24 or TypeScript	A private context is disposed automatically at the end of the block
`maxRetries` per request	Playwright 1.46 and newer	Retries connection resets only, so real 5xx bugs still fail the test
Testing WebMCP tools and passkeys	Playwright 1.61 to 1.64	End to end tests for AI tool calls in the browser and passwordless login
Tags and annotations in test details	Playwright 1.42 and newer	`test('...', { tag: '@smoke' })` and linked issues in the report

Lesson: What is new in Playwright Test, and how it helps at work

TIPS AND TRAPS

IN THE EXAM — If your UI tests already use Playwright, API tests can live in the same repo and CI job. Start by replacing slow UI setup steps with API calls.

Lesson: Why test APIs with Playwright Test

REMEMBER — API only projects never need browsers. Skip `npx playwright install` until you add UI tests.

Lesson: Project setup and config

COMMON MISTAKE — `test.describe.serial` runs the tests in order and stops after the first failure. Use it only for true step by step flows. Independent tests with fixtures, shown next, are better in most cases.

Lesson: Create, update and delete (CRUD)

REMEMBER Use `toMatchObject` when the API may add new fields, and `toEqual` when the exact shape is part of the contract.

Lesson: Assertions: matchers, soft checks and polling

REMEMBER Every test has its own data and cleans up after itself. Tests can run in any order and in parallel without breaking each other.

Lesson: Custom fixtures for tokens and test data

COMMON MISTAKE Without `send: 'always'`, `httpCredentials` waits for a 401 challenge. Restful Booker answers 403, so the password would never be sent.

Lesson: Authentication and negative tests

REMEMBER For larger sets, read a JSON or CSV file at the top of the spec with `fs.readFileSync` and loop over its rows in the same way.

Lesson: Data driven tests

REMEMBER The second test checks the checker. A schema that never fails is as useless as no schema.

Lesson: JSON Schema checks with Ajv

IN THE EXAM `waitForResponse` lets a UI test assert on the exact request the page sent. This catches bugs where the screen looks right but the wrong data reaches the server.

Lesson: UI and API together

COMMON MISTAKE Attachments and traces include request headers. If your API uses an `Authorization` header, avoid attaching raw headers, or replace the token with *** before you attach.

Lesson: Real project essentials: environments, secrets, test data, logging and cleanup

COMMON MISTAKE Inside Playwright Test, do not call `api.tracing.start()` yourself when `trace` is set in the config. The runner already records every request context the test creates, and a second start fails with `Tracing has been already started`. Manual tracing is for scripts and other languages that do not use the runner.

Lesson: What is new in Playwright Test, and how it helps at work

REMEMBER Upgrade Playwright on a branch, run the suite with `--repeat-each 3` to spot new flakiness, and read the release notes for changed defaults before you merge.

Lesson: What is new in Playwright Test, and how it helps at work

3.3 JavaScript · fetch

The 17 lessons of the JavaScript · fetch track.

KEY IDEAS

- **Why go native** An API test sends a request and checks the response. With Node 22 or newer you can do this with no packages at all, which means no supply chain risk, no version upgrades and instant installs in CI.
- **Assertions with node:assert** For older Node versions, a tiny helper gives the same partial match.
- **Hooks, shared login and auth checks** before runs once before the tests in its block, beforeEach before every test, and the after versions clean up. Use them to log in once and to create and delete test data.
- **Mocking fetch and time** When you test your own code that calls an API, replace fetch with a mock. The test controls every answer and checks every call, with no network.
- **Concurrency and a quick latency check** Sort n times from fastest to slowest. The p95 sits at
- **Framework design with the client** The BookingClient from the mocking lesson keeps tests short. The same class is tested with mocks and then used for real calls.

- **Real project essentials: environments, secrets, fake data, logging and cleanup** Node.js now ships most of what a professional API suite needs: .env loading, random ids, test hooks for cleanup, and a built in way to watch every fetch call without wrapping it. Here is how a real team uses them.
- **Real world scenario: a hotel front desk day** The receptionist logs in, books a room, adds breakfast, and the guest cancels. Someone without a login also tries to cancel. Using t.test subtests, each step appears as its own line in the report.

FORMULAS

Terminal

```
node --version                          # 22 or newer
npm test                                # runs every *.test.js file
node --test tests/crud.test.js          # one file
node --test --watch                     # rerun on save
BASE_URL=http://localhost:3001 npm test
node --env-file=.env --test             # load variables from .env
```

Lesson: Setup and running tests

Other auth styles

```
headers: { Authorization: `Bearer ${process.env.API_TOKEN}` }
headers: { 'x-api-key': process.env.API_KEY }
headers: { Authorization: 'Basic ' + btoa(`${user}:${password}`) }
```

Lesson: Hooks, shared login and auth checks

Runner options worth knowing

```
node --test --test-name-pattern="smoke"
    # only tests whose name matches
node --test --test-skip-pattern="slow"
    # skip by name
node --test --test-concurrency=4
    # files in parallel
node --test --test-reporter=tap
    # tap, spec, dot, junit, lcov
node --test --experimental-test-coverage
    # coverage of src/ used by tests
node --test --test-timeout=30000
    # fail any test over 30 s
```

Lesson: Reporters, coverage, filters and CI

Check a redirect without following it

```
const res = await fetch(`${BASE}/old-path`, { redirect: 'manual' });
assert.equal(res.status, 301);
assert.equal(res.headers.get('location'), '/new-path');
```

Lesson: Expert topics

.env.example (commit this, never the real .env)

```
TEST_ENV=local
BASE_URL=http://localhost:3001
API_USER=admin
API_PASSWORD=change-me
LOG_HTTP=1
```

Lesson: Real project essentials: environments, secrets, fake data, logging and cleanup

TABLES

Table: need · built in answer

Need	Built in answer
Send HTTP requests	`fetch`, `Request`, `Response`, `Headers`
Time limits	`AbortSignal.timeout(ms)`
Run tests	`node --test`, `test`, `describe`, `it`
Setup and cleanup	`before`, `after`, `beforeEach`, `afterEach`
Assertions	`node:assert/strict`
Mocks	`mock.method`, `mock.fn`, `mock.timers`
Reports and coverage	`--test-reporter`, `--experimental-test-coverage`
Secrets from a file	`node --env-file=.env`

Lesson: Why go native

Table: body type · content-type set automatically

Body type	Content-Type set automatically
`JSON.stringify(obj)` (a string)	text/plain, so set application/json yourself
`URLSearchParams`	application/x-www-form-urlencoded
`FormData`	multipart/form-data with a boundary
`Blob` or `Buffer`	the Blob type, or none

Lesson: Building requests: URL, headers and bodies

Table: need · built in answer

Need	Built in answer
Different servers	`BASE_URL` read in `src/config.js`
Secrets out of Git	`--env-file-if-exists=.env` locally, CI secrets in pipelines
Unique test data	`crypto.randomUUID()` and `crypto.randomInt()`
See every call	`node:diagnostics_channel` with the undici channels
Clean up	`after()` in the file, or `t.after()` inside one test

Lesson: Real project essentials: environments, secrets, fake data, logging and cleanup

Table: feature · since · how it helps an api tester

Feature	Since	How it helps an API tester
Snapshot assertions `t.assert.snapshot()`	Node 22.3, no flag needed since 22.13 and 23.4	Save a known good response once, then catch any unexpected change in later runs
`t.assert` on the test context	Node 22.2	Assertions tied to the test, so plans and snapshots know which test they belong to
Type stripping, run `.ts` directly	on by default in Node 22.18 and 24	Write typed tests and scripts with no build step
`URLPattern` global	Node 24	Match and extract ids from `Location` headers and links cleanly
Node 24 LTS	October 2025	Long term support until 2028. A safe target for CI images
`--env-file-if-exists`	Node 22.9	Load `.env` locally without failing in CI where the file does not exist

Lesson: What is new in Node.js for API tests, and how it helps at work

TIPS AND TRAPS

IN THE EXAM fetch works the same in Node and in the browser console. Testers can try a request in DevTools, then paste the same lines into a test file.
Lesson: Why go native

COMMON MISTAKE Forgetting `Content-Type: application/json` is the most common fetch mistake. Many servers then ignore the body or answer 400 or 500.
Lesson: Building requests: URL, headers and bodies

REMEMBER Inside one `describe`, tests run one after another by default, so the shared `id` is safe here. Different files run in parallel in separate processes.
Lesson: Create, update and delete (CRUD)

REMEMBER For full JSON Schema support add Ajv, the one package most native suites accept. The Playwright track shows how.
Lesson: Assertions with node:assert

COMMON MISTAKE Node's fetch does not keep cookies between calls like a browser does. Send the `Cookie` header yourself, or read `res.headers.getSetCookie()` and store the values.
Lesson: Hooks, shared login and auth checks

REMEMBER `{ concurrency: true }` runs the tests inside that `describe` at the same time. Use it only for tests that do not share state.
Lesson: Data driven tests

REMEMBER `mock.timers.enable({ apis: ['setTimeout'] })` fakes the clock, so retry and polling code with long waits can be tested instantly with `mock.timers.tick(ms)`.
Lesson: Mocking fetch and time

REMEMBER There is no `npm install` step at all. That is the main benefit of the native approach.
Lesson: Reporters, coverage, filters and CI

REMEMBER Logging through diagnostics channels also catches calls made by libraries you did not write, such as an SDK your client uses. That is often where the surprising requests are.
Lesson: Real project essentials: environments, secrets, fake data, logging and cleanup

COMMON MISTAKE Snapshot the **shape** or stable fields, not whole bodies with ids and timestamps. A snapshot that changes on every run trains the team to update it without reading it.
Lesson: What is new in Node.js for API tests, and how it helps at work

STEP 4 TypeScript

4.1 TypeScript · Playwright

The 16 lessons of the TypeScript · Playwright track.

KEY IDEAS

- **Why write API tests in TypeScript** Imagine a team of ten testers sharing one test suite for a payments API. The backend renames amount to amountPaise .
- **Typed models: interfaces, utility types and satisfies** Put the shapes of your API in one file. Every test imports them, so a change to the API is a change in one place.
- **Run time checks with Zod: one schema, one type** A type tells the editor what you expect. A Zod schema checks what actually arrived. Better still, Zod can produce the type from the schema, so you write the shape once and get both.

- **A typed client class with generics** A client class hides URLs and headers, so tests read like the business story. With types and Zod, every method returns data that is both checked and typed.
- **Typed fixtures for clients and test data** test.extend takes two type parameters: fixtures created for each test, and fixtures created once per worker. The editor then knows exactly what each test can ask for.
- **Types generated from the OpenAPI contract** Hand written types can drift away from the real API. In most companies the backend publishes an OpenAPI file, and the test project generates its types from it.
- **Framework design: environments, logging and a type gate** Real projects run the same tests against local, QA and staging, keep secrets out of the code, and print useful details when something fails. A small typed config file covers all three.
- **Real world scenario: a hotel front desk day** The receptionist logs in, books a room, adds breakfast, and the guest later cancels. Someone without a login also tries to cancel.

FORMULAS

Terminal

```
                  npm init -y
     npm install -D @playwright/test@1.64.1
    typescript@7.0.2 zod@4.6.7 @types/node@24
npx playwright test                          # run
                   every test
  npx tsc --noEmit                             #
        type check only, writes no files
  npx playwright test --grep @smoke            #
                tagged tests only
  BASE_URL=http://localhost:3001 npx playwright
                      test
```

Lesson: Project setup: packages, tsconfig and config

tests/client.spec.ts

```
import { test, expect } from '@playwright/test';
    import { BookingClient, ApiError } from
            '../src/booking-client';
 import { sampleBooking } from '../src/models';

test('create, find, patch and delete through the
       client', async ({ request }) => {
              const api = await new
         BookingClient(request).login();
     const { bookingid } = await api.create({
    ...sampleBooking, firstname: 'Typed' });

    expect(await api.find({ firstname: 'Typed'
           })).toContain(bookingid);
  expect((await api.patch(bookingid, { lastname:
    'Client' })).lastname).toBe('Client');
                  expect((await
   api.remove(bookingid)).status()).toBe(201);
                       await
expect(api.get(bookingid)).rejects.toThrow(ApiErr
                      or);
                       });
```

Lesson: A typed client class with generics

tests/fixtures.spec.ts

```
 import { test, expect } from '../src/fixtures';

   test('read a fresh booking', async ({ api,
                booking }) => {
                  expect((await
api.get(booking.id)).firstname).toBe(booking.data
                   .firstname);
                       });

 test('patch with the shared worker token', async
       ({ request, booking, token }) => {
                const res = await
    request.patch(`/booking/${booking.id}`, {
       headers: { Cookie: `token=${token}` },
     data: { additionalneeds: 'Late checkout' },
                        });
            await expect(res).toBeOK();
                       });
```

Lesson: Typed fixtures for clients and test data

src/export-schema.ts (run with: node src/export-schema.ts)

```
            import { z } from 'zod';
  import { BookingSchema } from './schemas.ts';

console.log(JSON.stringify(z.toJSONSchema(Booking
              Schema), null, 2));
```

Lesson: Latest TypeScript and tool features in real work

TABLES

Table: what typescript adds · what it means in daily work

What TypeScript adds	What it means in daily work
Types for requests and responses	Autocomplete for every field, no more guessing names from the docs
Errors before run time	`tsc --noEmit` catches typos, missing fields and wrong types in seconds
Safe refactoring	Rename a field in one type and the editor updates or flags every use
Typed fixtures and clients	A test cannot ask for a fixture that does not exist, or pass a booking where an id is expected
Types from the contract	Generate types from the OpenAPI file so tests and API docs never drift apart

Lesson: Why write API tests in TypeScript

Table: utility · gives you · used for

Utility	Gives you	Used for
`Partial<T>`	every field optional	PATCH bodies
`Pick<T, K>`	only the listed fields	summaries and list items
`Omit<T, K>`	every field except the listed ones	a create body without the server made `id`
`Readonly<T>`	fields that cannot be reassigned	shared test data nobody should change by accident
`Record<K, V>`	an object with known keys	lookup tables such as status code to message

Lesson: Typed models: interfaces, utility types and satisfies

Table: feature · since · how it helps an api tester

Feature	Since	How it helps an API tester
Native compiler written in Go	TypeScript 7.0, July 2026	Type checks are up to about ten times faster on big projects, so the type gate in CI no longer slows the pipeline. `strict` is on by default.
`satisfies`	TypeScript 4.9	Check test data against a type and keep exact literal values, as in `sampleBooking`.
`using` and `await using`	TypeScript 5.2	Dispose a request context automatically at the end of a block, even when an assertion throws.
`erasableSyntaxOnly`	TypeScript 5.8	Keeps you to syntax that can simply be stripped, which is what lets Node.js run `.ts` files directly.
Type stripping in Node.js	Node.js 22.18 and 24, on by default	`node scripts/seed.ts` runs a typed data seeding script with no build step.
Zod 4	2025	Faster parsing, smaller bundles, `z.toJSONSchema()` to share schemas with other teams.

Lesson: Latest TypeScript and tool features in real work

TIPS AND TRAPS

COMMON MISTAKE Types exist only while you write and check code. They are removed before the code runs, so they cannot check what the server really sent. That is why this track also uses **Zod**, which checks response bodies at run time.

Lesson: Why write API tests in TypeScript

IN THE EXAM Most new Playwright projects in companies are started in TypeScript. If you can read and write typed tests, you can join those teams on day one.

Lesson: Why write API tests in TypeScript

REMEMBER API only projects never need browsers. Skip `npx playwright install` until you add UI tests.

Lesson: Project setup: packages, tsconfig and config

REMEMBER `satisfies Booking` checks the object matches the type but keeps its exact literal types. A misspelt field or a string price is flagged at once, and `sampleBooking.firstname` is still known to be exactly `"Rakesh"`.

Lesson: Typed models: interfaces, utility types and satisfies

REMEMBER `import { type Booking }` marks an import as type only. It is removed when the file runs, which keeps run time imports clean and is required by some fast TypeScript runners.

Lesson: Create, update and delete (CRUD) with types

REMEMBER The third test checks the checker. A schema that never fails is as useless as no schema.

Lesson: Run time checks with Zod: one schema, one type

REMEMBER `checked()` is generic: give it `BookingSchema` and the return type is a booking, give it `TokenSchema` and it is `{ token: string }`. One helper, every endpoint, no casts.

Lesson: A typed client class with generics

REMEMBER Ask for a fixture that does not exist, such as `async ({ bookng })`, and the type checker flags it at once. In JavaScript you would only find out when the test runs and the value is `undefined`.

Lesson: Typed fixtures for clients and test data

REMEMBER `as const satisfies readonly (keyof Booking)[]` means the list can only hold real field names. If the API renames `totalprice`, this line turns red instead of silently testing a field that no longer exists.

Lesson: Data driven tests with typed case tables

REMEMBER Labelled tuple types such as `[name: string, headers: Record<string, string>]` document each column of a case table and show the labels in editor hints.

Lesson: Authentication and negative tests

COMMON MISTAKE openapi-typescript 7.13 still lists TypeScript 5 as its peer dependency, so installing it next to TypeScript 7 fails with a peer dependency error. Running it with `npx openapi-typescript@7.13.0` keeps it out of your project and avoids the clash. Checking peer dependencies like this is normal work when a big new compiler version arrives.

Lesson: Types generated from the OpenAPI contract

IN THE EXAM Add the generate step to CI and fail the build if `src/generated/booker.ts` changed but was not committed. That way nobody can change the API contract without the tests noticing.

Lesson: Types generated from the OpenAPI contract

REMEMBER When you upgrade TypeScript, Playwright or Zod at work, do it on a branch, run `tsc --noEmit` and the full suite, and read the release notes for removed options. TypeScript 7 turns some old, deprecated settings into hard errors.

Lesson: Latest TypeScript and tool features in real work